

INFORME TÉCNICO

Sistema de Ascensor Inteligente Controlado con ESP32

Justificación técnica, arquitectura, componentes y funciones del microcontrolador

Proyecto: Ascensor con ESP32

Fecha: septiembre 2026

1. Introducción

El presente informe describe el diseño técnico de un sistema de ascensor inteligente para un edificio de baja a mediana altura (2 a 6 niveles), controlado mediante un microcontrolador ESP32. El documento justifica la selección de este microcontrolador frente a otras alternativas, detalla los componentes electromecánicos y electrónicos que conforman el sistema, y describe las funciones y periféricos internos del ESP32 que serán aprovechados en la implementación.

El objetivo del proyecto es desarrollar un sistema de control de ascensor capaz de gestionar el llamado de piso, el posicionamiento de la cabina, la apertura y cierre de puertas, la señalización visual y sonora, y opcionalmente el monitoreo remoto del estado del equipo a través de una conexión inalámbrica, todo esto con un solo módulo de bajo costo y alto rendimiento.

2. Justificación del uso del ESP32

La selección del microcontrolador es una decisión crítica en un sistema de control como el de un ascensor, ya que este debe manejar múltiples tareas de forma simultánea (lectura de sensores, control de motor, interfaz de usuario y, opcionalmente, comunicación en red) manteniendo tiempos de respuesta confiables. A continuación se comparan las alternativas más comunes para este tipo de proyecto.

Criterio	ESP32	Arduino UNO/Nano	Raspberry Pi	PLC comercial
----------	-------	------------------	--------------	---------------

Núcleos / velocidad	Dual-core 32 bits, hasta 240 MHz	1 núcleo 8 bits, 16 MHz	Multi-núcleo, alto rendimiento	Variable, orientado a ciclo de escaneo
Conectividad inalámbrica	WiFi y Bluetooth integrados	No (requiere módulo externo)	WiFi integrado en algunos modelos	Requiere módulo adicional
Entradas/salidas (GPIO)	Hasta 34 pines multipropósito	Limitadas (~20)	Limitadas y sin ADC nativo robusto	Amplias pero de alto costo
Entradas analógicas (ADC)	Hasta 18 canales, 12 bits	6 canales, 10 bits	No posee ADC nativo	Módulos externos
PWM por hardware	Hasta 16 canales independientes	Limitado (6 pines)	Software/limitado	Sí, con mayor costo
Sistema operativo / multitarea	FreeRTOS (tiempo real, dual-core)	No (secuencial)	Linux (no determinístico en tiempo real)	Determinístico, propietario
Costo aproximado	Bajo (5–12 USD)	Bajo	Medio-alto (35–75 USD)	Alto (100+ USD)
Consumo energético	Bajo, con modos de bajo consumo	Bajo	Medio-alto	Medio
Curva de aprendizaje / comunidad	Amplia comunidad, IDE Arduino y ESPIDF	Amplia comunidad	Amplia comunidad	Requiere software propietario

En función de la comparación anterior, el ESP32 resulta la opción más adecuada para este proyecto por las siguientes razones:

- **Suficiencia de E/S:** cuenta con pines digitales, analógicos, PWM e interrupciones suficientes para manejar sensores de posición, finales de carrera, botonera de piso, sensores de puerta y actuadores, sin necesidad de expansores de puertos.
- **Procesamiento en tiempo real y multitarea:** al ejecutar FreeRTOS sobre dos núcleos, permite dedicar un núcleo al control del motor y la lectura de sensores (tareas críticas en tiempo) y otro a la interfaz de usuario y la comunicación en red, sin que una tarea bloquee a la otra.
- **Conectividad nativa:** el WiFi integrado permite implementar monitoreo remoto, registro de eventos, notificación de fallas y un panel de control (dashboard) sin hardware adicional, algo que en un Arduino requeriría un módulo externo y en un PLC representaría un costo elevado.
- **Relación costo-beneficio:** ofrece un desempeño cercano al de soluciones industriales a una fracción del costo, lo cual es apropiado para un proyecto académico o un prototipo funcional que pueda escalar a producción.
- **Bajo consumo y modos de ahorro de energía:** permite mantener el sistema en espera de forma eficiente mientras el ascensor no está en uso.
- **Ecosistema de desarrollo maduro:** compatible con el IDE de Arduino, PlatformIO y ESP-IDF, con amplia documentación y librerías para motores, pantallas, sensores y protocolos de comunicación (I2C, SPI, UART, MQTT).

3. Arquitectura general del sistema

El sistema se organiza en cuatro subsistemas principales, todos coordinados por el ESP32 como unidad central de control:

- Subsistema de tracción: motor y mecanismo de desplazamiento vertical de la cabina.
- Subsistema de sensado: detección de posición/piso, límites de recorrido y estado de la puerta.
- Subsistema de interfaz de usuario: botonera de llamado, botonera interna de cabina, pantalla e indicadores.
- Subsistema de comunicación y seguridad: conectividad WiFi/MQTT, paro de emergencia y protecciones eléctricas.

El flujo de control es el siguiente: el usuario solicita un piso (desde el pasillo o desde dentro de la cabina); el ESP32 determina la dirección de desplazamiento necesaria; activa el driver del motor en el sentido correspondiente; monitorea continuamente los sensores de piso mediante interrupciones; al detectar el piso objetivo, detiene el motor, acciona la apertura de puerta, actualiza la pantalla y los indicadores, y —si la conectividad está disponible— publica el nuevo estado en el servidor o dashboard remoto.

4. Componentes del proyecto

A continuación se listan los componentes de hardware propuestos para el prototipo, junto con su función dentro del sistema.

Componente	Modelo / tipo sugerido	Función en el sistema
Microcontrolador	ESP32-WROOM-32 (DevKit v1)	Unidad central de control, procesamiento y comunicación.
Componente	Modelo / tipo sugerido	Función en el sistema
Motor de tracción	Motorreductor DC 12–24 V o motor paso a paso NEMA 17/23	Genera el movimiento vertical de la cabina mediante polea y cable o cremallera.
Driver de motor	BTS7960 (motor DC) o A4988/TB6600 (paso a paso)	Controla dirección y velocidad del motor a partir de señales PWM/digitales del ESP32.
Sensores de piso	Sensores infrarrojos ranurados o interruptores final de carrera (limit switch)	Detectan la posición de la cabina en cada nivel para detener el motor con precisión.
Sensor de puerta	Sensor magnético tipo reed switch	Verifica el estado abierto/cerrado de la puerta antes de permitir el movimiento.
Actuador de puerta	Micro servomotor o motorreductor pequeño	Abre y cierra automáticamente la puerta de la cabina.

Pantalla	LCD 16x2 o display OLED, con módulo I2C	Muestra el piso actual, dirección de movimiento y mensajes de estado.
Botonera de llamado	Pulsadores por piso	Permiten solicitar la cabina desde cada nivel.
Botonera de cabina	Pulsadores internos	Permiten seleccionar el piso destino desde dentro de la cabina.
Indicadores	LEDs de dirección y buzzer	Señalización visual y sonora de movimiento, llegada y alarmas.
Módulo relé	Relé de 1 a 2 canales	Activa cargas de mayor corriente, como el motor de la puerta o alimentación auxiliar.
Fuente de alimentación	Fuente conmutada 12/24 V para motores + regulador a 3.3/5 V para lógica	Alimenta de forma independiente la etapa de potencia y la etapa de control.
Botón de paro de emergencia	Pulsador normalmente cerrado (NC)	Corta la alimentación del motor ante una condición de riesgo.
Cableado y estructura mecánica	Poleas, cable de acero o correa, guías y contrapeso	Soporte mecánico y transmisión del movimiento de la cabina.

5. Funciones y periféricos del ESP32 utilizados

El ESP32 no se emplea únicamente como un simple controlador de entradas y salidas: el proyecto aprovecha varios de sus periféricos y capacidades internas de forma específica, como se detalla a continuación.

Función / periférico	Uso específico en el proyecto
GPIO digital (entradas/salidas)	Lectura de finales de carrera, botoneras de piso y cabina; activación de relés, LEDs y buzzer.
PWM (LEDC)	Control de velocidad del motor DC de tracción (aceleración y desaceleración suaves) y posicionamiento del servo de puerta.
ADC (convertor analógico-digital)	Lectura de sensores analógicos opcionales, como sensor de corriente del motor para detectar atascos o sobrecarga.
Función / periférico	Uso específico en el proyecto
Interrupciones externas (GPIO ISR)	Detección inmediata del paso por cada sensor de piso y del pulsador de paro de emergencia, sin necesidad de sondeo constante (polling).
Comunicación I2C	Control del display LCD/OLED que muestra el estado del ascensor.
Comunicación UART	Depuración durante el desarrollo y, opcionalmente, enlace con módulos adicionales.

FreeRTOS (multitarea en dual-core)	Ejecución simultánea de la tarea de control de motor/sensores (tiempo crítico) en un núcleo y la tarea de interfaz/comunicación en el otro, evitando bloqueos entre ellas.
WiFi (modo estación)	Conexión a la red local para el envío de telemetría y recepción de comandos remotos.
Protocolo MQTT / HTTP	Publicación del estado del ascensor (piso actual, dirección, alarmas) hacia un panel de monitoreo o servidor, y recepción de llamadas remotas.
Timers por hardware	Control de tiempos de espera de puerta abierta, tiempo máximo de desplazamiento entre pisos (detección de fallas) y anti-rebote (debounce) de pulsadores.
Memoria no volátil (NVS)	Almacenamiento de configuración, como el piso de referencia (home) y parámetros calibrados, incluso tras un corte de energía.
Modo de bajo consumo (light sleep)	Reducción del consumo del módulo mientras el ascensor permanece inactivo, sin perder la capacidad de respuesta a un llamado.

6. Lógica general de funcionamiento

El algoritmo de control implementado en el ESP32 sigue, de forma simplificada, la siguiente secuencia:

- 1. Inicialización de periféricos (GPIO, PWM, I2C, WiFi) y lectura del piso actual almacenado en memoria no volátil.
- 2. Espera de una solicitud, ya sea desde la botonera de piso, la botonera de cabina o un comando remoto vía MQTT.
- 3. Verificación de que la puerta esté cerrada y no exista una condición de paro de emergencia activa.
- 4. Cálculo de la dirección de movimiento y activación del driver del motor mediante PWM.
- 5. Monitoreo mediante interrupciones de los sensores de piso hasta detectar el nivel objetivo.
- 6. Detención del motor, actualización de pantalla e indicadores, y apertura de la puerta.
- 7. Publicación del nuevo estado hacia el sistema de monitoreo remoto (si está disponible).
- 8. Temporización de espera y cierre automático de la puerta, retornando al estado de espera.

7. Consideraciones de seguridad

Dado que el sistema involucra elementos mecánicos en movimiento, se contemplan las siguientes medidas de seguridad a nivel de diseño:

- Corte físico de alimentación del motor mediante un pulsador de emergencia independiente del software.
- Límites de recorrido redundantes (sensores de piso más finales de carrera de fin de trayecto) para evitar que la cabina exceda el rango permitido.
- Bloqueo por software del movimiento si la puerta no se detecta cerrada.
- Temporizador de vigilancia (watchdog) para reiniciar el sistema ante un bloqueo inesperado del firmware.

- Detección de sobrecorriente en el motor como indicio de atasco mecánico, deteniendo el movimiento de forma preventiva.

8. Conclusión

El ESP32 constituye una solución técnica y económicamente adecuada para el control de un ascensor inteligente a escala de prototipo o de edificio de baja altura. Su combinación de procesamiento dual-core con FreeRTOS, periféricos de E/S suficientes (PWM, ADC, interrupciones, I2C, UART) y conectividad inalámbrica integrada permite implementar, en un solo módulo, tanto el control determinístico del motor y los sensores como la capa de monitoreo remoto, sin necesidad de hardware adicional significativo. Esto reduce la complejidad del sistema, el costo total del proyecto y el tiempo de desarrollo, manteniendo un nivel de confiabilidad adecuado para una aplicación de este tipo.